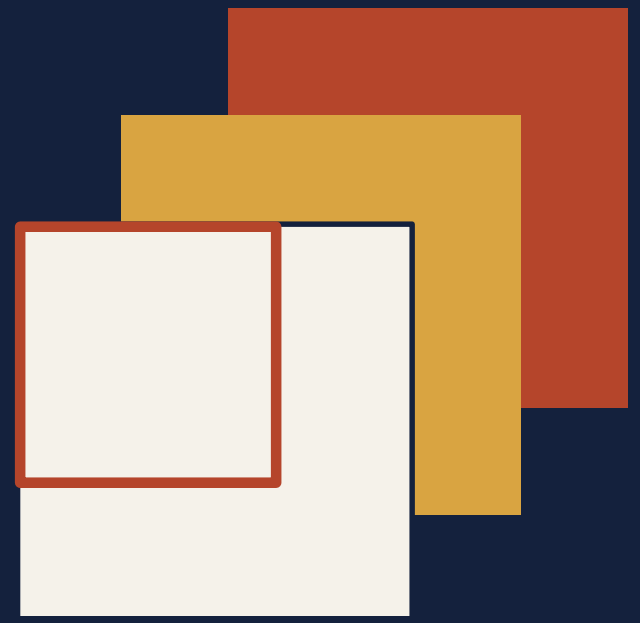
CNN을 코드로 확인하기



손으로 계산한 합성곱·풀링·파라미터 수를 Keras로 재현하고, Fashion-MNIST 분류기로 그 원리를 확인한다

INTRO

오늘의 실습 순서

절	내용	강의 슬라이드	시간
0	환경 준비 · 텐서 모양 규칙	4	2분
1	합성곱 손계산을 코드로 검증 · 출력 크기 공식	8, 10	4분
2	필터는 특징 탐지기: 엣지 필터	9	2.5분
3	풀링과 이동 불변성	13, 14	2.5분
4	데이터 준비 (Fashion-MNIST)	19	1.5분
5	CNN 구축 · 파라미터 수 추적	19, 20	3분
6	학습과 평가	19	2.5분
7	MLP와 비교 · 이동 실험	5, 14	3.5분
8	학습된 필터와 특징 맵	15	2분
–	정리 · 과제 안내	–	1.5분

0 · 환경 준비

시작 전에 두 가지만 확인하자

① GPU 런타임

Colab 메뉴
런타임 → 런타임 유형 변경 → T4 GPU

첫 셀을 실행해 **GPU:** 줄에 장치가 보이면 준비 완료. CPU에서도 실행은 되지만 학습이 몇 배 느리다.

그래프 제목은 한글 깨짐을 피하려고 영어로 적었다.

② Keras의 텐서 모양

이미지 `(B, H, W, C)`
필터 `(F, F, C_in, K)`

채널이 마지막에 온다. 흑백 28×28도 **(28, 28, 1)**로 채널 축을 붙여야 한다.

강의 20쪽 표의 (B, C, H, W)는 PyTorch 표기. 파라미터 수는 같다.

손계산 결과를 이중 for문으로 재현한다

```
def conv2d_naive(x, w, b=0, stride=1):
    N, F = x.shape[0], w.shape[0]
    O = (N - F) // stride + 1
    y = np.zeros((O, O))
    for i in range(O):
        for j in range(O):
            patch = x[i*stride:i*stride+F,
                      j*stride:j*stride+F]  # 겹치기
            y[i, j] = np.sum(patch * w) + b  # 곱하고 더하기
    return y

# 같은 계산을 TensorFlow로
tf.nn.conv2d(x.reshape(1,5,5,1),
             w.reshape(3,3,1,1), 1, "VALID")
```

두 결과가 같아야 한다

```
[[ 0, -2,  2],
 [ 0, -2,  1],
 [ 1,  0,  0]]
```

손계산과 같은가? True

필터를 **뒤집지 않는** 교차상관이라는 점을 코드에서 확인하자. **patch * w**를 그대로 곱한다.

1 · 출력 크기 공식

공식으로 예측하고, Keras로 확인한다

$$O = \lfloor (N + 2P - F) / S \rfloor + 1$$

사례	N	F	P	S	공식	Keras
손계산 예제	5	3	0	1	3	3
same 패딩	5	3	1	1	5	5
LeNet-5 C1	32	5	0	1	28	28
ResNet 첫 층	224	7	3	2	112	112

Keras의 패딩

"valid" = P 0

"same" = S 1일 때 출력 크기 유지

그 밖의 P는 **ZeroPadding2D(P)**를 앞에 붙인다

ResNet: 223/2 = 111.5 → 내림 111 → +1

필터는 경계에서만 크게 반응한다

숫자 예제 재현

10 10 10 200 200 200
(세 행이 같음)

↓ 수직 엣지 필터 [1, 0, −1] × 3행

[0, −570, −570, 0]

3 × (10 − 200) = −570. 음수는 **어두움** → **밝음** 방향의 경계.

실제 의류 이미지에 적용

앵클부츠 한 장에 수직·수평 엣지 필터를 각각 적용해 세 장을 나란히 그린다.

관찰 ① 배경과 신발 안쪽처럼 평평한 곳은 0(흰색)

관찰 ② 수직 필터는 세로 윤곽, 수평 필터는 밑창 같은 가로 윤곽

관찰 ③ 빨강(+)·파랑(−)이 경계의 방향을 알려 준다

풀링의 불변성은 창 안에서만 성립한다

MaxPooling2D(2) · AveragePooling2D(2)

최대 `[[6, 5], [3, 7]]`
평균 `[[3.5, 2], [1.5, 3.5]]`
파라미터 수 `0`

강의 13쪽 4×4 특징 맵 그대로

입력	값	최대 풀링	A와 비교
A 기준	[0, 9, 0, 0]	[9, 0]	–
B ← 1칸	[9, 0, 0, 0]	[9, 0]	같음 · 불변
C → 1칸	[0, 0, 9, 0]	[0, 9]	달라짐

C는 창 경계를 넘었다. 불변성은 국소적·근사적이다 → 7절에서 실제 이미지로 다시 확인

4 · 데이터 준비

정규화하고 채널 축을 붙인다

load_data() 직후

(60000, 28, 28)

uint8 · 0~255

→

÷ 255.0

(60000, 28, 28)

float32 · 0~1

→

[..., np.newaxis]

(60000, 28, 28, 1)

Conv2D 입력 준비 완료

0 T-shirt	1 Trouser	2 Pullover	3 Dress	4 Coat
5 Sandal	6 Shirt	7 Sneaker	8 Bag	9 Ankle boot

훈련 60,000장 · 테스트 10,000장 · 28×28 흑백 · 10종 의류

특징 추출부 두 블록, 분류부 두 층

```
keras.Sequential([
  layers.Input(shape=(28, 28, 1)),
  layers.Conv2D(32, 3, padding="same",
            activation="relu"), # 28×28×32
  layers.MaxPooling2D(2),         # 14×14×32
  layers.Conv2D(64, 3, padding="same",
            activation="relu"), # 14×14×64
  layers.MaxPooling2D(2),         # 7×7×64
  layers.Flatten(),               # 3136
  layers.Dense(128, activation="relu"),
  layers.Dense(10),               # logits
])
```

공간은 줄고 채널은 는다

28 → 14 → 7 (풀링)
1 → 32 → 64 (필터 개수)

마지막 층은 로짓

소프트맥스는 손실 함수의 **from_logits=True**가 함께 처리한다

5 · 파라미터 수 추적

summary()의 숫자를 공식으로 맞춰 본다

층	Keras 출력 모양	계산식	파라미터
Conv2D 1→32	(None, 28, 28, 32)	(9·1+1)·32	320
MaxPooling2D	(None, 14, 14, 32)	–	0
Conv2D 32→64	(None, 14, 14, 64)	(9·32+1)·64	18,496
MaxPooling2D	(None, 7, 7, 64)	–	0
Flatten	(None, 3136)	–	0
Dense 3136→128	(None, 128)	(3136+1)·128	401,536
Dense 128→10	(None, 10)	(128+1)·10	1,290
합계			421,642

95%

가 완전연결층 **하나**에 있다

합성곱 두 층은 합쳐서 2만 개도 안 된다 — 가중치 공유

5 에폭 학습 후 무엇을 볼 것인가

학습 설정

손실 SparseCategoricalCrossentropy (from_logits=True)
Adam 1e-3 · 배치 64
5 에폭 · 검증 10%

학습 곡선

train과 val 손실이 함께 내려가는가? 둘의 간격이 벌어지기 시작하면 과적합의 신호

오답 8장

T = 정답, P = 예측
셔츠·티셔츠·풀오버·코트처럼 사람도 헷갈리는 쌍이 많다

테스트 정확도는 대체로 **90% 안팎**. 시드를 고정해도 GPU 연산 때문에 소수점 아래는 실행마다 조금씩 다를 수 있다.

7 · MLP와 비교

파라미터는 CNN이 오히려 더 많다

MLP

`Flatten → Dense 128 → Dense 10`

101,770

(784+1)·128 + (128+1)·10

CNN

`[Conv → Pool] × 2 → Dense 128 → Dense 10`

421,642

그중 401,536이 Dense 3136→128

강의 5쪽의 “파라미터 폭발”과 모순일까? 범인은 합성곱이 아니라 **Flatten 뒤의 완전연결층**이다. 28×28 작은 입력이라 MLP도 아직 작을 뿐, 224×224×3이면 MLP는 1.5억 개가 된다.

7 · 이동 실험

테스트 이미지를 오른쪽으로 0~4픽셀 옮기면?

실행 전에 예측해 보자

① 두 모델 중 어느 쪽 정확도가 더 가파르게 떨어질까?

② CNN은 4픽셀을 옮겨도 정확도가 그대로일까?

결과를 읽는 법

MLP는 픽셀 하나하나가 고정된 가중치를 가져 이동에 약하다.

CNN도 **완전히 불변은 아니다**. 풀링은 두 번뿐이고, 마지막 **Flatten + Dense**가 다시 위치에 민감하기 때문.

```
out[:, :, dx:, :] = x[:, :, :-dx, :]  # 빈 칸은 0(검정)으로
```

CNN 안에서 실제로 무엇이 학습되었나

학습된 필터 32개

```
cnn.layers[0].get_weights()
→ (3, 3, 1, 32)
```

2절에서 직접 만든 엣지 필터와 닮은 것이 섞여 있다. 사람이 값을 넣지 않았는데도 **역전파가 찾아낸** 필터다.

층별 특징 맵

```
conv1 (1, 28, 28, 32)
conv2 (1, 14, 14, 64)
```

conv1 원래 모양이 보이고 윤곽·밝기 대비에 반응

conv2 해상도는 낮고, 엣지를 조합한 더 추상적인 패턴

WRAP-UP

오늘 코드로 확인한 네 가지

01 손계산한 합성곱·출력 크기·풀링 결과가 Keras와 **정확히 일치**했다.

02 파라미터 (F·F·C_in+1)·K는 이미지 크기와 무관하고, 이 모델 파라미터의 **95%는 완전연결층**에 있다.

03 CNN은 MLP보다 이동에 강하지만 **완전히 불변은 아니다**.

04 필터는 사람이 정하지 않고 **데이터로부터 학습**된다.

과제

1. 합성곱 필터 개수 변경 : 첫번째 합성곱 층의 필터 개수를 32개에서 16개로 변경하고 model.summary() 를 실행하고, 첫번째 합성곱 층의 출력 모양과 전체 파라미터 수를 필터 32개와 비교하여라.
2. 합성곱 필터 크기 변경 : 최초 모델에서 합성곱 층의 필터의 크기를 5x5로 변경하여 위와 같이 비교하여라
3. 완전연결층의 뉴런 수 변경 : 완전연결층의 뉴런 수를 128개에서 64개로 변경하여 위와 같이 파라미터 수를 비교하여라.